Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы оптимизации и управления»

ОТЧЕТ
к лабораторной работе №5
на тему:
«ДВОЙСТВЕННЫЙ СИМПЛЕКС-МЕТОД»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил доцент каф. ИКТ
ГОРБУКОВ Дмитрий Михайлович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Реализовать двойственный симплекс-метод для прямой задачи линейного программирования в канонической форме. По заданным c, A, b и начальному упорядоченному набору базисных индексов B необходимо либо получить оптимальный план прямой задачи, либо установить ее несовместность.

При разработке отчета и программы учитывался алгоритм из методички лабораторной работы: вычисление базисного допустимого плана двойственной задачи, формирование псевдоплана, выбор отрицательной базисной компоненты, расчет значений mu_j и sigma_j и обновление базиса.

2 ВЫПОЛНЕНИЕ РАБОТЫ

В программе реализованы процедуры обращения базисной матрицы и двойственный симплекс-метод без использования библиотеки NumPy. На каждой итерации строятся матрица A_B^-1, вектор c_B, двойственный план y и псевдоплан kappa.

Для примера из методических указаний рассматривается задача с вектором c = [-4, -3, -7, 0, 0], матрицей A = [[-2, -1, -4, 1, 0], [-2, -2, -2, 0, 1]], вектором b = [-1, -3/2] и начальным базисом B = [4, 5]. На первой итерации получены y = [0, 0] и kappa = [0, 0, 0, -1, -1.5].

Поскольку псевдоплан содержит отрицательные базисные компоненты, метод выбирает индекс j2 = 5, затем строит Delta_y = [0, 1], вычисляет mu_1 = mu_2 = mu_3 = -2 и значения sigma_1 = 2, sigma_2 = 1.5, sigma_3 = 3.5. Минимум достигается для x2, поэтому на следующей итерации базис становится B = [4, 2].

На второй итерации получены y = [0, 1.5] и псевдоплан kappa = [0, 0.75, 0, -0.25, 0]. После вычисления Delta_y = [1, -0.5], значений mu_1 = -1, mu_3 = -3, mu_5 = -0.5 и оценок sigma_1 = 1, sigma_3 = 4/3, sigma_5 = 3 в базис вводится переменная x1. После обновления базиса получаем B = [1, 2].

На третьей итерации псевдоплан принимает вид kappa = [0.25, 0.5, 0, 0, 0]. Все его компоненты неотрицательны, следовательно найден оптимальный план прямой задачи x = [1/4, 1/2, 0, 0, 0]. Результат работы программы приведен ниже на рисунке 2.1.

```
--- Итерация 2 ---
Текущий базис B = [4, 2]
A_B^-1 =
[1.0, -0.5]
[-0.0, -0.5]
c_B = [0.0, -3.0]
Двойственный план y = [0.0, 1.5]
Псевдоплан kappa = [0.0, 0.75, 0.0, -0.25, 0.0]
Отрицательная базисная компонента: kappa_4 = -0.25
Delta_y = [1.0, -0.5]
mu_1 = -1.0
sigma_1 = 1.0
mu_3 = -3.0
sigma_3 = 1.333333
mu_5 = -0.5
sigma_5 = 3.0
Минимальное sigma0 = 1.0
В базис входит x1, из базиса выходит x4

--- Итерация 3 ---
Текущий базис B = [1, 2]
A_B^-1 =
[-1.0, 0.5]
[1.0, -1.0]
c_B = [-4.0, -3.0]
Двойственный план y = [1.0, 1.0]
Псевдоплан kappa = [0.25, 0.5, 0.0, 0.0, 0.0]
Псевдоплан неотрицателен. Найден оптимальный план прямой задачи.

--- Результат ---
Оптимальный план x = [0.25, 0.5, 0.0, 0.0, 0.0]
```

Рисунок 2.1 – Результат работы программы

ВЫВОД

В ходе лабораторной работы реализован двойственный симплекс-метод для решения задачи линейного программирования в канонической форме. На примере из методических указаний метод за три итерации построил неотрицательный псевдоплан и получил оптимальное решение x = [0.25, 0.5, 0, 0, 0]. Это подтверждает корректность вычисления двойственного плана, псевдоплана и правил замены базисных индексов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Python Documentation [Электронный ресурс]. – Режим доступа: https://docs.python.org/. – Дата доступа: 09.04.2026.

[2] NumPy Documentation [Электронный ресурс]. – Режим доступа: https://numpy.org/doc/. – Дата доступа: 09.04.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация двойственного симплекс-метода

```
TOL = 1e-9

def identity_matrix(n):
    return [[1.0 if i == j else 0.0 for j in range(n)] for i in
range(n)]


def inverse_matrix(matrix):
    n = len(matrix)
    aug = [matrix[i][:] + identity_matrix(n)[i] for i in range(n)]

    for col in range(n):
        pivot = col
        while pivot < n and abs(aug[pivot][col]) < TOL:
            pivot += 1
        if pivot == n:
            raise ValueError("Матрица вырождена.")
        if pivot != col:
            aug[col], aug[pivot] = aug[pivot], aug[col]

        pivot_value = aug[col][col]
        for j in range(2 * n):
            aug[col][j] /= pivot_value

        for row in range(n):
            if row == col:
                continue
            factor = aug[row][col]
            if abs(factor) < TOL:
                continue
            for j in range(2 * n):
                aug[row][j] -= factor * aug[col][j]

    return [row[n:] for row in aug]


def get_submatrix_columns(matrix, columns):
    return [[row[j] for j in columns] for row in matrix]


def matrix_vector_mul(matrix, vector):
    return [sum(matrix[i][j] * vector[j] for j in range(len(vector)))
for i in range(len(matrix))]


def row_vector_matrix_mul(vector, matrix):
    cols = len(matrix[0])
    rows = len(matrix)
    return [sum(vector[i] * matrix[i][j] for i in range(rows)) for j
in range(cols)]
```

```
def dot(a, b):
    return sum(x * y for x, y in zip(a, b))


def column(matrix, j):
    return [matrix[i][j] for i in range(len(matrix))]


def round_vector(vector, digits=6):
    return [round(x, digits) for x in vector]


def print_vector(name, vector):
    print(f"{name} = {round_vector(vector)}")


def print_matrix(name, matrix):
    print(f"{name} =")
    for row in matrix:
        print(round_vector(row))


def dual_simplex_method(c, a, b, basis, verbose=True):
    """
    Двойственный симплекс-метод.

    Returns:
        optimal_x или None, если прямая задача несовместна.
    """
    m = len(a)
    n = len(a[0])
    basis = basis[:]
    iteration = 1

    while True:
        if verbose:
            print(f"\n--- Итерация {iteration} ---")
            print(f"Текущий базис B = {[idx + 1 for idx in basis]}")

        a_b = get_submatrix_columns(a, basis)
        a_b_inv = inverse_matrix(a_b)
        c_b = [c[j] for j in basis]
        y = row_vector_matrix_mul(c_b, a_b_inv)

        kappa_b = matrix_vector_mul(a_b_inv, b)
        kappa = [0.0] * n
        for pos, basis_index in enumerate(basis):
            kappa[basis_index] = kappa_b[pos]

        if verbose:
            print_matrix("A_B^-1", a_b_inv)
            print_vector("c_B", c_b)
            print_vector("Двойственный план y", y)
            print_vector("Псевдоплан kappa", kappa)
```

```
        negative_positions = [i for i in range(m) if kappa_b[i] < -
TOL]
        if not negative_positions:
            if verbose:
                print("Псевдоплан неотрицателен. Найден оптимальный
план прямой задачи.")
            return [0.0 if abs(value) < TOL else value for value in
kappa]

        k = min(negative_positions, key=lambda idx: (kappa_b[idx],
basis[idx]))
        leaving_basis_index = basis[k]
        delta_y = a_b_inv[k][:]

        if verbose:
            print(f"Отрицательная базисная компонента:
kappa_{leaving_basis_index + 1} = {round(kappa_b[k], 6)}")
            print_vector("Delta_y", delta_y)

        candidates = []
        for j in range(n):
            if j in basis:
                continue
            mu_j = dot(delta_y, column(a, j))
            if verbose:
                print(f"mu_{j + 1} = {round(mu_j, 6)}")
            if mu_j < -TOL:
                sigma_j = (c[j] - dot(column(a, j), y)) / mu_j
                candidates.append((sigma_j, j, mu_j))
                if verbose:
                    print(f"sigma_{j + 1} = {round(sigma_j, 6)}")

        if not candidates:
            if verbose:
                print("Для всех небазисных индексов mu_j >= 0. Прямая
задача несовместна.")
            return None

        sigma0, j0, _ = min(candidates, key=lambda item: (item[0],
item[1]))
        if verbose:
            print(f"Минимальное sigma0 = {round(sigma0, 6)}")
            print(f"В базис входит x{j0 + 1}, из базиса выходит
x{leaving_basis_index + 1}")

        basis[k] = j0
        iteration += 1


if __name__ == "__main__":
    # Пример из lab5_conditions.pdf
    c_example = [-4.0, -3.0, -7.0, 0.0, 0.0]
    a_example = [
        [-2.0, -1.0, -4.0, 1.0, 0.0],
        [-2.0, -2.0, -2.0, 0.0, 1.0],
    ]
    b_example = [-1.0, -1.5]
    basis_example = [3, 4]
```

```
    result = dual_simplex_method(c_example, a_example, b_example,
basis_example, verbose=True)
    if result is None:
        print("\nОтвет: прямая задача несовместна.")
    else:
        print("\n--- Результат ---")
        print_vector("Оптимальный план x", result)
```